UNITÀ 2 · WEB SERVICE · LEZIONE 12

Usare REST da JavaScript

L'ultimo client di BiblioAPI è il più diffuso del mondo: il browser. fetch, JSON, una pagina che si aggiorna da sola, e il misterioso errore CORS.

5G TPSI fetch, async/await, DOM, CORS e preflight, sicurezza delle chiavi

PER COMINCIARE

Hai mai fatto una richiesta HTTP da JavaScript? Con che cosa?

SONDAGGIO

- **A** fetch
- **B** XMLHttpRequest
- **C** Una libreria, come axios o jQuery
- **D** Me l'ha scritto un'AI
- **E** Mai fatto

UN PO' DI STORIA

Dalle pagine che si ricaricano alle pagine che chiedono dati

1999 · XMLHttpRequest

Microsoft lo crea per la webmail di Outlook in Internet Explorer 5. Poi lo copiano tutti i browser.

2005 · "Ajax"

Jesse James Garrett dà un nome alla tecnica che fa funzionare Gmail e Google Maps.

2015 · fetch

Un'interfaccia moderna, basata sulle Promise, arriva nei browser principali.

> **Non dichiaro di aver inventato JSON.**
> **L'ho scoperto: esisteva già in natura.**
>
> **Douglas Crockford**, che diede il nome a JSON e ne scrisse la specifica, nel suo intervento *The JSON Saga*

FETCH IN SEI RIGHE

Chiedi, aspetta, controlla, leggi

```
const BASE = "http://localhost:8000/api/libri";

async function carica() {
  const r = await fetch(BASE);
  if (!r.ok) throw new Error("HTTP " + r.status);
  const libri = await r.json();
  console.log(libri[0].titolo);
}

carica().catch(err => console.error(err));
```

- `fetch` restituisce subito una **Promise** della risposta.
- `await` aspetta senza bloccare la pagina.
- `r.ok` è vero per i codici da 200 a 299.
- `r.json()` legge il corpo e lo trasforma in oggetti: è anch'essa asincrona.

QUIZ A TRABOCCHETTO

fetch su `/api/libri/9999`, che risponde 404. Che cosa succede alla Promise?

QUIZ

A Viene rifiutata e si finisce nel catch

B Resta in attesa per sempre

C Si risolve: r.ok è falso e r.status vale 404

D Si risolve con `null`

AL COMPUTER · 20 minuti

Il meteo della tua città in una pagina

```
const url = "https://api.open-meteo.com/v1/forecast" +
  "?latitude=41.90&longitude=12.50&current=temperature_2m,wind_speed_10m";
async function meteo() {
  const r = await fetch(url);
  if (!r.ok) throw new Error("HTTP " + r.status);
  const d = await r.json();
  document.getElementById("t").textContent =
    d.current.temperature_2m + " " + d.current_units.temperature_2m;
}
meteo().catch(e => document.getElementById("t").textContent = e.message);
```

1. Crea **meteo.html** con un `<p id="t">` e questo script; aprilo nel browser.
2. Metti le coordinate della tua città e mostra anche il vento.

Controllo: la pagina mostra un numero seguito da `°C`, per esempio `17.4 °C`, e il vento in `km/h`. Nella scheda Rete compare una richiesta con stato 200 verso `api.open-meteo.com`.

DAL JSON ALLA PAGINA

textContent, non innerHTML

Così sì

```
for (const l of libri) {
  const li = document.createElement("li");
  li.textContent = `${l.titolo} (${l.anno})`;
  ul.append(li);
}
```

Il titolo resta testo, qualunque carattere contenga.

Così no

```
for (const l of libri) {
  ul.innerHTML +=
    `<li>${l.titolo}</li>`;
}
```

Un titolo con `<img src=x onerror=…>` esegue codice nel browser di chi guarda.

AL COMPUTER · 20 minuti

BiblioAPI nel browser... e il primo errore

1. Avvia BiblioServer della lezione 9 così com'è.
2. Crea **libri.html** con un `<ul id="elenco">`, un `<p id="msg">` e la funzione `carica()` che riempie la lista con `textContent`.
3. Apri la pagina con Live Server (o con doppio clic).
4. Apri la Console (F12) e leggi l'errore. Poi guarda la scheda Rete: la richiesta è partita?

Controllo: la lista resta vuota e la Console dice che l'accesso è stato **bloccato dalla CORS policy**: manca l'header `Access-Control-Allow-Origin`. Il messaggio d'errore in pagina è `Failed to fetch` (in Firefox: `NetworkError…`).

SAME-ORIGIN POLICY E CORS

Il browser protegge l'utente, non il server

Origine = **schema + host + porta**. Una pagina può leggere liberamente solo le risposte della propria origine.

Pagina su `http://localhost:5500` chiama…	**Stessa origine?**
`http://localhost:5500/api`	sì
`http://localhost:8000/api`	no: porta diversa
`https://localhost:5500/api`	no: schema diverso
`http://127.0.0.1:5500/api`	no: host diverso

CORS

Cross-Origin Resource Sharing: il **server** dichiara con degli header quali altre origini possono leggere le sue risposte. Il **browser** controlla e decide.

curl e Java non sono browser: il CORS non li riguarda.

METTITI ALLA PROVA

La GET bloccata dal CORS: il server l'ha eseguita?

QUIZ

A Sì: la richiesta è arrivata e il server ha risposto, ma il browser non passa la risposta allo script

B No: il browser non l'ha mai inviata

C No: il server l'ha rifiutata con 403

D Sì, e lo script riceve una risposta vuota con stato 0

IL SERVER DÀ IL PERMESSO

Quattro header e una risposta a OPTIONS

```
static void cors(HttpExchange ex) {
  Headers h = ex.getResponseHeaders();
  h.set("Access-Control-Allow-Origin", "*");   // in produzione: l'origine precisa
  h.set("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE");
  h.set("Access-Control-Allow-Headers", "Content-Type");
  h.set("Access-Control-Expose-Headers", "Location");
}

// all'inizio di gestisci(), prima di tutto il resto:
cors(ex);
if (ex.getRequestMethod().equals("OPTIONS")) {   // la richiesta di preflight
  ex.sendResponseHeaders(204, -1); ex.close(); return;
}
```

PUZZLE

Un POST con JSON da un'altra origine: metti in ordine

1. Il browser invia prima una richiesta OPTIONS con l'header Origin
2. Il server risponde 204 con gli header Access-Control-Allow-*
3. Il browser controlla che origine, metodo e header siano ammessi
4. Il browser invia il vero POST con il JSON
5. Il server risponde 201 e il browser passa la risposta allo script
6. Lo script chiama fetch con POST e Content-Type application/json

PUZZLE

Abbina ogni header al suo significato

Origin	---	I metodi ammessi, nella risposta al preflight
Access-Control-Allow-Origin	---	Gli header che lo script può inviare, come Content-Type
Access-Control-Allow-Methods	---	Gli header della risposta che lo script può leggere
Access-Control-Allow-Headers	---	Il server dice quale origine può leggere la risposta
Access-Control-Expose-Headers	---	Il browser dice da quale origine viene la pagina che chiama

AL COMPUTER · 35 minuti

La biblioteca nel browser: leggi, aggiungi, elimina

```
const r = await fetch(BASE, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ titolo: d.get("titolo"),
    autore: d.get("autore"), anno: Number(d.get("anno")) })
});
msg.textContent = r.ok ? "Aggiunto: " + r.headers.get("Location")
                       : "Errore " + r.status;
```

1. Aggiungi `cors()` e il ramo OPTIONS al server e riavvialo.
2. In libri.html: un form con titolo, autore e anno che invia il POST; poi richiama `carica()`.
3. Accanto a ogni libro un pulsante **Elimina** con `method: "DELETE"`.

Controllo: la lista mostra i 3 libri. Aggiungendo Lessico famigliare compare `Aggiunto: /api/libri/4` e il libro in lista. Nella scheda Rete il POST è preceduto da un **OPTIONS** con stato 204. Elimina toglie il libro senza ricaricare la pagina.

QUIZ PER CHI HA FINITO IL LABORATORIO

Il POST risponde 201 con Location, ma `r.headers.get("Location")` dà `null`. Perché?

QUIZ

A fetch non legge mai gli header della risposta

B Bisogna scrivere "location" in minuscolo

C Il server ha risposto 201 senza Location

D Da un'altra origine lo script legge solo pochi header, se il server non ne espone altri

PONTI VERSO OGGI

Un chatbot nella tua pagina, senza regalare la chiave

Browser

fetch verso `/api/chat` del tuo server, senza chiavi.

Il tuo server

Aggiunge la chiave presa da una variabile d'ambiente e controlla i limiti.

API del modello

Riceve il POST con JSON e risponde.





BIGLIETTO D'USCITA

Completa prima di uscire

Per fetch un 404 non è un errore: bisogna controllare la proprietà _____.

Un'origine è formata da schema, host e _____.

Prima di un PUT verso un'altra origine il browser invia una richiesta _____.

L'header che permette a una pagina di leggere le risposte è Access-Control-Allow-_____.

Per inserire un titolo nella pagina in modo sicuro si usa _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- fetch + await, controlla sempre r.ok
- JSON diventa oggetti JavaScript con r.json()
- Dati esterni nella pagina con textContent
- Origine = schema + host + porta
- CORS: il server permette, il browser controlla
- Le chiavi delle API restano sul server

> **All'esame**: BiblioAPI è un'architettura completa da raccontare all'orale: un servizio REST in Java, tre client diversi (Java, JSP, browser), una collezione di test e le scelte di sicurezza.

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso.